SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

ERROR ANALYSIS TASK

Name of the Student: K. Vannur Swamy

Reg. No: 192525265

Course Name: Deep Learning **Course Code:** MLA0403

Faculty: Dr. Arul Raj A.M

Total Marks: 25

1. Error Analysis in Maximum Likelihood Estimation (MLE)

a) Identify the conceptual error in the likelihood formulation.

Answer: The conceptual error is treating the likelihood as a sum of probabilities. For independent observations, the likelihood is the **product** of the probabilities of all observed samples. Maximizing a sum does not represent the probability of the complete observed dataset.

b) Explain how this mistake affects parameter estimation.

Answer: Using a sum gives an incorrect objective function. Therefore, the estimated parameters do not maximize the probability of the observed data. Even with a large dataset, the model can converge to poor or biased parameter values because it is optimizing the wrong quantity.

c) Suggest the correct mathematical formulation and reasoning.

Answer: For independent observations, the correct likelihood is:
$\mathbf{L(\theta) = \prod_{i=1}^{n} P(y_i \mid x_i, \theta)}$
The parameter estimate is θ■ = arg $\max_{\theta}$ L(θ). The product combines the probability of every observation and therefore represents the likelihood of the complete dataset.

d) How would using log-likelihood fix computational issues?

Answer: Taking logarithms converts the product into a sum: $\mathbf{\log L(\theta) = \Sigma \log P(y_i \mid x_i, \theta)}$. This avoids numerical underflow caused by multiplying many small probabilities and makes optimization easier while preserving the maximizing parameter because log is a monotonically increasing function.

2. Error Diagnosis in Perceptron Learning

a) List possible implementation errors in the learning algorithm.

Answer: Possible errors include an incorrect prediction rule, wrong target-label encoding, incorrect weight-update equation, updating weights with the wrong sign, forgetting the bias term, using an inappropriate stopping condition, and failing to update weights only for misclassified samples.

b) Analyze how incorrect learning rate or weight updates affect convergence.

Answer: If the learning rate is too small, weight changes are tiny and convergence can be very slow. If it is too large, the weights may overshoot the required values and training can oscillate. An incorrect update direction or formula can prevent the classifier from reaching a separating boundary.

c) Identify dataset-related issues that may cause this problem.

Answer: The dataset may not actually be linearly separable, labels may be incorrect or inconsistently encoded, features may have extreme scales or outliers, or preprocessing may have introduced errors. Duplicate or contradictory samples can also make convergence difficult.

d) Propose modifications to ensure convergence.

Answer: Verify labels and the update rule, include the bias correctly, choose a suitable learning rate, normalize or standardize features, shuffle training samples, and use a clear stopping criterion. For a truly linearly separable dataset, the standard perceptron update should converge after these issues are corrected.

3. Backpropagation Gradient Error Analysis

a) Identify possible mathematical or coding errors in gradient computation.

Answer: Common errors include an incorrect derivative of the activation function, wrong chain-rule order, incorrect matrix dimensions, missing transposition, wrong sign in the update, using gradients from the wrong layer, or accidentally computing a gradient of the loss with respect to the wrong variable.

b) Explain the role of activation functions in gradient flow.

Answer: Activation functions determine how gradients pass through each layer. Their derivatives control the gradient magnitude. Saturating functions can produce very small derivatives, while suitable non-saturating activations can help maintain useful gradients during training.

c) Analyze how vanishing or exploding gradients affect training.

Answer: Vanishing gradients become extremely small as they propagate backward, so earlier layers learn very slowly. Exploding gradients become extremely large, causing unstable updates and possibly increasing the loss. Both problems can prevent effective training.

d) Suggest techniques to fix the issue (e.g., normalization, initialization).

Answer: Use suitable weight initialization such as Xavier/Glorot or He initialization, normalize inputs and use normalization layers when appropriate, select suitable activation functions, use an appropriate learning rate, and apply gradient clipping when gradients become too large.

4. Gradient Descent vs SGD Error Investigation

a) Explain why batch gradient descent is slow in large datasets.

Answer: Batch Gradient Descent computes the gradient using the entire training dataset for every update. With a large number of samples, each update requires substantial computation and memory, so parameter updates occur less frequently and training can be slow.

b) Analyze the cause of fluctuations in SGD.

Answer: SGD updates parameters using one randomly selected sample at a time. Different samples produce different gradient estimates, so the update direction contains noise. This causes the loss to fluctuate instead of decreasing smoothly.

c) Compare the error behavior of Mini-Batch Gradient Descent.

Answer: Mini-Batch Gradient Descent uses a small group of samples for each update. Its gradient is less noisy than pure SGD but cheaper than full-batch computation. Therefore, the loss usually decreases more smoothly than SGD while still allowing frequent updates.

d) Recommend the best approach and justify with error trade-offs.

Answer: For large datasets, Mini-Batch Gradient Descent is generally a good practical choice. It provides a balance between the stable but slow updates of Batch Gradient Descent and the fast but noisy updates of SGD. Batch GD is useful for smaller datasets, while SGD can be useful when frequent inexpensive updates are preferred.

5. Curse of Dimensionality Error Analysis

a) Explain how high dimensionality increases model error.

Answer: As the number of features increases, the feature space becomes much larger and data become sparse. A fixed dataset provides fewer examples for each region of this space, making it harder for a model to learn reliable patterns and increasing the risk of overfitting.

b) Identify signs of overfitting in this scenario.

Answer: A major sign is high training performance combined with poor test performance. A large gap between training and validation/test error indicates that the model has learned training-specific patterns instead of general relationships.

c) Analyze why distance-based learning fails in high dimensions.

Answer: In high-dimensional spaces, distances between points tend to become less informative because points can appear similarly far apart. This reduces the usefulness of nearest-neighbor and other distance-based methods and makes local similarity harder to identify.

d) Suggest dimensionality reduction or regularization techniques to improve performance.

Answer: Dimensionality can be reduced using PCA or feature selection. Regularization methods such as L1 or L2 penalties can reduce model complexity. Other useful approaches include removing irrelevant features, collecting more representative data, and using cross-validation to select model settings.